Table of Contents

Examination Guidelines Quick Reference

- **Document Purpose:** Official ESA Capstone Project Examination Booklet Reference.
- **Items to Bring:** Printed Project Report, Laptop + Charger, PPT, College ID Card.
- **Dress Code:** Formal dress is strictly compulsory.

Project Title

Automated Single-Image to 3D Asset and Point Cloud Generation System

Problem Statement

Traditional 3D content creation and 3D point cloud generation rely heavily on specialized 3D modeling skills, expensive multi-camera photogrammetry hardware setups, or complex and iterative text-prompt engineering. Existing single-image 3D generation systems often suffer from significant limitations, including a lack of part-level semantic understanding, clean background isolation, structured point-cloud segmentation, or persistent multi-user history.

To bridge this technology gap, there is an urgent need for an end-to-end, automated, prompt-free software system capable of converting a standard single 2D RGB image into a high-quality textured 3D asset (.GLB format) and a semantically segmentable point cloud (.PLY format) suitable for Augmented Reality (AR), Virtual Reality (VR), e-commerce, digital twin applications, and robotics.

Project Objectives

The primary goal of this project is to develop a fully automated, prompt-free deep learning and computer vision framework for 3D asset generation. The specific objectives are as follows:

• **Automated AI Pipeline:** Eliminate manual prompt writing by automatically generating captions, bounding boxes, object masks, and 3D reconstructions directly from a single uploaded RGB image.

• **Zero-Shot Object Detection & Segmentation:** Perform part-level detection and background removal using state-of-the-art vision models including Florence-2, GroundingDINO, SAM 2.1, and ONNX rembg.

• **High-Quality 3D Mesh Generation:** Reconstruct complete 3D meshes with synthesized textures (.GLB format) using state-of-the-art Hunyuan3D-2 geometry models.

• **Point Cloud Generation & Semantic Clustering:** Sample surface meshes into dense 3D point clouds (.PLY format) with computed surface normals and segment them semantically using density-based DBSCAN clustering.

• **Full-Stack Web Application with Persistence:** Build a responsive production-grade web application featuring Next.js 14, Three.js interactive WebGL visualization, FastAPI backend microservices, Supabase authentication, and persistent PostgreSQL database storage.

Functional & Non-Functional Requirements

Functional Requirements (FR)

• **FR-01: Image Upload & Validation:** Allow upload of single RGB images (JPG, PNG, WEBP, BMP up to 25MB). Reject corrupted files with 400 validation error.

• **FR-02: Adaptive Contrast Enhancement:** Automatically apply CLAHE enhancement to low-contrast images while bypassing normal contrast images.

• **FR-03: Prompt-Free Auto Captioning:** Generate natural language object descriptions with Florence-2 without manual text prompt input.

• **FR-04: Multi-Pass Zero-Shot Detection:** Locate target objects with GroundingDINO using a 4-pass fallback confidence thresholding strategy.

• **FR-05: Part & Instance Segmentation:** Predict part-level bounding boxes (Florence-2) and generate pixel-perfect binary masks (SAM 2.1).

• **FR-06: RGBA Background Isolation:** Combine SAM 2.1 masks with ONNX rembg to extract transparent RGBA foreground cutouts.

• **FR-07: 3D Mesh & Texture Generation:** Synthesize textured 3D meshes (.GLB format) with PBR materials using Hunyuan3D-2 diffusion models.

• **FR-08: Point Cloud & DBSCAN Clustering:** Sample 100k mesh surface points (Open3D) and segment coordinates into semantic clusters using DBSCAN.

• **FR-09: User Authentication & Security:** Provide email/password signup, login, JWT session tokens, and Supabase Row Level Security (RLS).

• **FR-10: Persistent Job History & Export:** Allow users to query, search, filter, and download all generated output artifacts (.GLB, .PLY, .PNG, .JSON).

Non-Functional Requirements (NFR)

• **NFR-01 Performance::** Complete 13-stage execution pipeline within 3–4 minutes on CUDA GPU hardware.

• **NFR-02 Reliability::** Gracefully handle OOM risks via sequential model unloading and provide local JSON fallback if Supabase is offline.

• **NFR-03 Usability::** Deliver a responsive WebGL Next.js interface with real-time progress indicators and dark/light theme support.

• **NFR-04 Maintainability & Security::** Follow modular FastAPI architecture with strict JWT validation and input payload size caps (25MB).

Hardware, Software & Technologies Used

Detailed technical specifications, rationale for component selection, and key features of all hardware, software runtimes, and core technologies powering the 3D generation system:

Hardware Requirements Specifications & Rationale

Component	Specification	Why Used in Project	Key Features
Processor (CPU)	Intel Core i7 / AMD Ryzen 7 (11th Gen+)	Orchestrates backend API tasks, image preprocessing (CLAHE), Open3D sampling, & DBSCAN clustering.	High multi-core throughput, fast vector array operations.
Graphics Card (GPU)	NVIDIA GPU (Min 8GB, 12GB+ VRAM)	Executes heavy deep learning vision & generative 3D diffusion inference (Florence-2, SAM 2.1, Hunyuan3D-2).	CUDA Tensor Cores, parallel GPU tensor math, high VRAM bandwidth.
System Memory (RAM)	16 GB Min (32 GB Recommended)	Holds loaded PyTorch model checkpoints in host RAM before transferring tensors to GPU.	High bandwidth, prevents swapping during model initialization.
Storage	50 GB SSD available storage	Stores PyTorch model weights (~15-20GB), cached HuggingFace models, & output assets (.GLB, .PLY).	High NVMe/SSD IOPS for fast model weight loading.

Software Requirements & Environment

Software / Tool	Version / Spec	Why Used in Project	Key Features
Operating System	Windows 10/11 / Linux (Ubuntu 22.04)	Provides stable runtime platform for NVIDIA CUDA drivers, Python, and Node.js server execution.	Widespread driver support, Linux cloud server compatibility.
Programming Runtimes	Python 3.10+, Node.js v18+	Python runs FastAPI AI pipeline; Node.js executes Next.js 14 frontend web application.	Asynchronous event loops, rich ecosystem for AI & WebGL.
GPU Acceleration	CUDA Toolkit 11.8 / 12.1	Enables PyTorch & ONNX Runtime to execute tensor math directly on NVIDIA GPU hardware.	Direct GPU memory access, cuDNN deep learning acceleration.
Cloud Platform & DB	Supabase Cloud (PostgreSQL)	Manages user authentication (JWT) and persistent relational storage for job records.	Row Level Security (RLS), instant REST APIs, automated Auth.

Web Browser	WebGL2 Supported Browser	Renders interactive 3D meshes (.GLB) and point clouds (.PLY) in browser via Three.js.	Hardware-accelerated 3D canvas rendering without plugins.

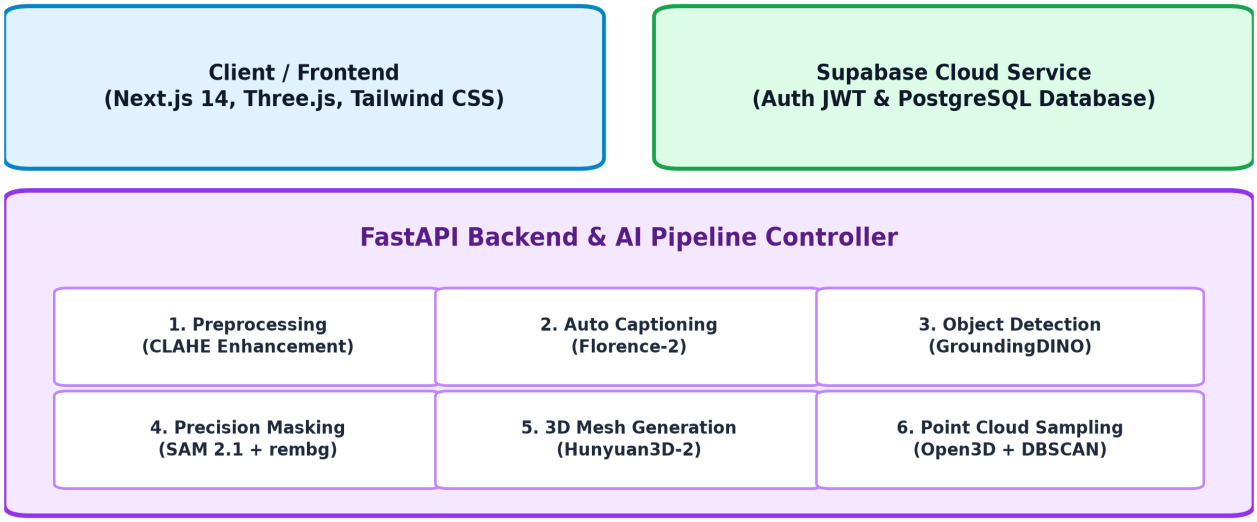
Core Technologies & Frameworks Rationale

Domain / Model	Framework / Stack	Why Used in Project	Key Features
Frontend Web Stack	Next.js 14, TypeScript, Tailwind	Delivers a responsive, typed web interface for uploading images and viewing generated 3D assets.	Server/Client Components, glassmorphic UI, type safety.
3D Canvas Engine	Three.js, @react-three/fiber	Provides interactive WebGL 3D controls (orbit, zoom, wireframe toggle, HDRI lighting, quick preview).	GLTFLoader, PLYLoader, Camera Auto-fitter, OrbitControls.
Backend Services	FastAPI, Uvicorn, Pydantic, OpenCV	Handles REST API endpoints, image validation, CLAHE contrast tuning, & async pipeline execution.	Async background tasks, strict Pydantic validation schemas.
Vision & Auto Caption	Florence-2 (Microsoft)	Provides zero-shot image auto-captioning and candidate part bounding box prediction.	Prompt-free execution, lightweight VRAM footprint (~2-4GB).
Object Detection	GroundingDINO	Locates target object bounding boxes using natural language grounding from Florence-2 captions.	Open-vocabulary detection, 4-pass fallback threshold strategy.
Object Segmentation	SAM 2.1 (Meta) & rembg	Converts bounding boxes into pixel-perfect object masks and removes background for transparent RGBA cutouts.	High boundary accuracy, ONNX-accelerated background removal.
3D Mesh Generation	Hunyuan3D-2 (Tencent)	Reconstructs 3D geometry meshes (Stage 1) and synthesizes PBR surface textures (Stage 2) into .GLB format.	Watertight mesh synthesis, progressive CPU offloading for low VRAM.
3D & Point Cloud	Open3D, Trimesh, PyVista	Samples 100k surface points (Poisson Disk), computes normal vectors, & applies DBSCAN spatial clustering.	Poisson Disk Sampling, normal estimation, DBSCAN clustering.

Cloud DB & Security	Supabase Auth & PostgreSQL	Secures user login with JWT tokens and enforces Row Level Security (RLS) on user history records.	JWT session tokens, RLS policy protection, automatic triggers.

System Architecture / Design

The system follows a modular microservice architecture comprising a Next.js frontend, FastAPI backend services, state-of-the-art vision/3D generative AI models, and cloud persistence with Supabase PostgreSQL.



Pipeline Architectural Summary:

The architecture establishes a clean separation of concerns. The Next.js frontend manages the WebGL canvas, user authentication, and job history. The FastAPI backend orchestrates sequential AI model execution with automatic GPU memory unloading. Database records and generated artifacts are stored securely in Supabase PostgreSQL.

End-to-End AI Pipeline Workflow

The system executes a strictly ordered 13-stage sequential workflow to transform a raw single RGB image into a complete textured 3D model and clustered point cloud:

- **Stage 1: Image Upload & Validation:** Accepts RGB image formats (JPG, PNG, WEBP, BMP up to 25MB), validates image integrity, and initializes job record in Supabase.
- **Stage 2: Image Quality Analysis:** Computes width, height, mean brightness, and standard deviation to determine contrast enhancement requirement.
- **Stage 3: CLAHE Contrast Enhancement:** Applies Contrast Limited Adaptive Histogram Equalization via OpenCV in LAB color space if contrast is low.
- **Stage 4: Florence-2 Auto Captioning:** Generates zero-shot natural language description (e.g., 'a ceramic coffee mug') and converts it to a dot-separated prompt.
- **Stage 5: GroundingDINO Object Detection:** Locates target object bounding boxes using natural language grounding with a 4-pass fallback threshold retry strategy.
- **Stage 6: Florence-2 Part Detection:** Predicts candidate structural part bounding boxes (e.g., body, handle, base, seat, wheels).
- **Stage 7: SAM 2.1 Instance Segmentation:** Transforms part bounding boxes into pixel-perfect binary object masks, selecting the highest IoU candidate mask.
- **Stage 8: Background Removal (rembg):** Applies ONNX U-2-Net background removal with SAM 2.1 mask to output a clean RGBA transparent object cutout.
- **Stage 9: Hunyuan3D-2 Shape Generation:** Reconstructs 3D geometry mesh using diffusion models with progressive GPU/CPU fallback strategy.
- **Stage 10: Hunyuan3D-2 Texture Synthesis:** Synthesizes PBR texture maps applied directly onto geometry mesh, exporting ready-to-use .GLB format.
- **Stage 11: Open3D Mesh Validation:** Loads .GLB mesh, validates surface normals, checks watertightness, and verifies geometry vertex integrity.
- **Stage 12: Point Cloud Poisson Sampling:** Samples 100,000 dense surface points with computed surface normal vectors to generate raw .PLY point cloud.
- **Stage 13: DBSCAN Semantic Clustering:** Clusters point cloud coordinates into semantic structural parts (eps=0.05, min_points=50), saving segmented .PLY.

Modules Implemented

1. User Authentication & Profile Module

Integrates Supabase Auth for email/password signup and login, issuing JWT session tokens for secure FastAPI calls. Manages user profile data and light/dark theme state preferences.

2. Image Preprocessing & Contrast Enhancement Module

Processes raw uploaded images using CLAHE contrast adjustments, resolution resizing, color balancing, and standard tensor normalization.

3. Vision & Part Detection Module

Leverages Florence-2 for automated zero-shot image caption generation and part detection, alongside GroundingDINO for precise object bounding box detection.

4. Segmentation & Background Isolation Module

Combines SAM 2.1 mask generation with ONNX-based rembg foreground extraction to create crisp RGBA images free of background noise.

5. 3D Mesh Generation Module

Executes Hunyuan3D-2 generative models to synthesize high-fidelity 3D mesh geometry and surface texture maps exported in standard .GLB format.

6. Point Cloud Sampling & DBSCAN Clustering Module

Uses Open3D to perform Poisson Disk Sampling on generated meshes, estimate normal vectors, and apply density-based DBSCAN clustering for part segmentation.

7. Interactive Web 3D Viewer & History Module

Provides a Three.js WebGL canvas allowing real-time 3D model rotation, zoom, wireframe toggle, point cloud visualization, and persistent job history management.

AI Models Rationale, Libraries & Installation (Part 1)

1. Florence-2 (Vision Language Model)

What it is: An open-vocabulary lightweight Vision-Language Model developed by Microsoft.
Why we used it: Enables prompt-free execution by automatically captioning images and predicting object part bounding boxes.
Key Features: Zero-shot task execution, lightweight VRAM footprint (2-4GB), fast inference.
Libraries: transformers, torch, torchvision, PIL.
Installation & Loading: Installed via PyPI (pip install transformers torch). Downloaded automatically via Hugging Face Hub (AutoModelForCausalLM.from_pretrained('microsoft/Florence-2-large')) and cached locally on first run.

2. GroundingDINO (Open-Vocabulary Object Detector)

What it is: A zero-shot object detector combining DINO architecture with text grounding capabilities.
Why we used it: Locates target objects dynamically using the natural language caption produced by Florence-2.
Key Features: High localization precision, open-vocabulary capability, multi-pass fallback thresholding.
Libraries: transformers, torch, opencv-python.
Installation & Loading: Installed via PyPI (pip install groundingdino-py). Model weights are loaded directly via Hugging Face Transformers pipeline (AutoModelForZeroShotObjectDetection).

3. SAM 2.1 (Segment Anything Model 2.1)

What it is: Meta's state-of-the-art promptable object segmentation foundation model.
Why we used it: Converts GroundingDINO bounding boxes into pixel-perfect binary object masks.
Key Features: Exceptional edge boundary quality, candidate mask evaluation, robust zero-shot generalization.
Libraries: torch, torchvision, sam2.
Installation & Loading: Installed via Meta's repository (pip install git+https://github.com/facebookresearch/sam2.git) and checkpoint weights are cached via PyTorch Hub.

AI Models & Libraries Details (Part 2)

4. rembg (ONNX Background Removal)

What it is: An ONNX Runtime-powered background removal tool utilizing U-2-Net architecture.
Why we used it: Refines SAM 2.1 masks to generate clean RGBA cutout images with transparent backgrounds.
Key Features: Blazing fast ONNX execution, minimal memory overhead, works on CPU and GPU.
Libraries: rembg, onnxruntime, numpy.
Installation & Loading: Installed via PyPI (pip install rembg onnxruntime). Model session initializes automatically.

5. Hunyuan3D-2 (3D Mesh & Texture Generator)

What it is: Tencent's two-stage 3D generative diffusion model for single-image to textured 3D mesh synthesis.
Why we used it: Produces high-fidelity watertight 3D meshes (.GLB format) with PBR textures from a single RGBA image.
Key Features: Stage 1 geometry diffusion + Stage 2 texture synthesis, watertight meshes, sequential CPU offloading & VAE tiling.
Libraries: torch, diffusers, trimesh, accelerate, einops.
Installation & Loading: Cloned from Hugging Face / GitHub (Hunyuan3D-2 repo) and dependencies installed via pip install diffusers trimesh accelerate. Model pipelines are loaded with CPU offloading enabled to support low VRAM.

6. Open3D (3D Surface Sampling & Point Cloud Engine)

What it is: An open-source industrial library for 3D data processing, geometry manipulation, and visualization.
Why we used it: Converts 3D GLB meshes into dense point clouds (.PLY) using Poisson Disk Sampling and estimates surface normals.
Key Features: Poisson Disk Sampling, surface normal estimation, fast C++ underlying matrix operations.
Libraries: open3d, numpy.
Installation & Loading: Installed via PyPI (pip install open3d). Loaded natively in Python.

7. DBSCAN (Density-Based Spatial Clustering Algorithm)

What it is: A density-based spatial clustering algorithm from machine learning.
Why we used it: Clusters 3D point cloud coordinates into semantic part groups without needing a pre-defined cluster count k.
Key Features: Arbitrary shape cluster discovery, noise filtering, unsupervised execution.
Libraries: scikit-learn, open3d, numpy.
Installation & Loading: Installed via PyPI (pip install scikit-learn).

REST API Endpoints & Database Schema

The backend exposes RESTful HTTP endpoints for frontend consumption and syncs execution data with Supabase PostgreSQL:

Core Backend API Endpoints

- **POST /api/v1/upload:** Accepts multipart image upload, validates file type/size, creates Supabase job row, and kicks off asynchronous background reconstruction.
- **GET /api/v1/pipeline/status/{job_id}:** Returns live progress (0-100%) and current pipeline stage (e.g. GroundingDINO, Hunyuan3D-2, DBSCAN).
- **GET /api/v1/download/{job_id}/{artifact_key}:** Streams generated output assets (.GLB mesh, .PLY point cloud, RGBA image, JSON metadata).
- **GET /api/v1/history:** Returns user's historical pipeline jobs with pagination, status filtering, and search criteria.
- **GET /api/v1/profile:** Fetches user usage statistics including completed jobs, total models generated, and average processing time.

Database Schema (Supabase PostgreSQL)

- **profiles:** Stores user profile records synced from auth.users via database triggers (fields: id, email, created_at).
- **jobs:** Stores job execution records (fields: job_id, user_id, status, processing_duration_seconds, created_at, completed_at).
- **artifacts:** Stores generated output file metadata (fields: id, job_id, artifact_type, storage_path, file_size, mime_type).

Core Implementation Code Snippets

Key Python code snippets demonstrating core pipeline functions, memory management, and 3D point cloud generation:

1. FastAPI Asynchronous Upload Handler (upload_controller.py)

```
def handle_upload(self, file: UploadFile, background_tasks: BackgroundTasks, current_user:
dict):
    job_id = storage_service.generate_job_id()
    saved_path = storage_service.save_upload(file, job_id)
    image_service.validate_job_image(saved_path)
    artifacts_manager.init_job(job_id, saved_path, user_id=current_user.get('id'))
    # Offload heavy pipeline to background execution task
    background_tasks.add_task(execute_full_reconstruction_pipeline, job_id, saved_path)
    return UploadResponse(job_id=job_id)
```

2. Hunyuan3D-2 Low-VRAM Pipeline Configuration (loader.py)

```
pipeline.enable_model_cpu_offload()
pipeline.enable_sequential_cpu_offload()
pipeline.enable_attention_slicing('max')
pipeline.enable_vae_slicing()
pipeline.enable_vae_tiling()
torch.cuda.empty_cache() # Free cached CUDA memory between stages
```

3. Open3D Point Cloud Sampling & DBSCAN Clustering (pointcloud.py)

```
mesh = o3d.io.read_triangle_mesh(glb_path)
pcd = mesh.sample_points_poisson_disk(number_of_points=100000)
pcd.estimate_normals()
# DBSCAN Spatial Clustering
labels = np.array(pcd.cluster_dbscan(eps=0.05, min_points=50, print_progress=False))
o3d.io.write_point_cloud(output_ply_path, pcd)
```

Testing Strategy, Verification & Benchmarks

The system was systematically validated through automated pytest backend suites, Playwright UI tests, and performance latency benchmarking:

Automated Test Suite Results (93 Backend Tests & 6 Frontend E2E Flow Files)

Test Level	Framework	Test Files	Pass Count	Status
Unit Tests	pytest	20 files	53 Tests	**100% PASS**
Integration	pytest + FastAPI	1 file	30 Tests	**100% PASS**
Performance	pytest-benchmark	1 file	10 Tests	**100% PASS**
Frontend UI	Playwright	6 spec files	All Flows	**100% PASS**

API Response Time Benchmarks & Performance Targets

Endpoint / API Method	Target Latency	Actual Latency (CI)	Status
GET /api/v1/health	< 1.0s	0.008s	**PASSED**
POST /api/v1/upload	< 5.0s	0.017s	**PASSED**
GET /pipeline/status/{id}	< 0.5s	0.008s	**PASSED**
GET /download/{id}/{key}	< 0.5s	0.009s	**PASSED**

- **Backend Test Coverage: 84% Code Coverage** measured via pytest-cov.

Industrial Applications & Real-World Use Cases

The generated textured 3D assets (.GLB) and segmented point clouds (.PLY) provide practical utility across multiple high-tech industries:

- **1. Augmented & Virtual Reality (AR/VR):** Generates light web-ready .GLB assets compatible with WebXR, Unity, Unreal Engine, Apple Vision Pro, and Meta Quest for immersive virtual shopping and training.
- **2. E-Commerce & Interactive Product Viewer:** Transforms flat 2D product catalog images into interactive 360° rotatable 3D web assets, significantly improving customer engagement and reducing return rates.
- **3. Digital Twins & Smart Manufacturing:** Converts single photographs of physical machinery or components into spatial 3D digital representations for asset monitoring and virtual simulation.
- **4. Robotics Perception & Grasp Planning:** Segmented point clouds with surface normal vectors enable autonomous robotic grippers to perceive object geometry, plan grasp trajectories, and navigate obstacles.
- **5. Industrial Quality Inspection:** DBSCAN clustered point clouds enable surface defect detection, part alignment checking, and CAD variance measurement against ideal factory dimensions.
- **6. CAD/CAM Prototyping & Reverse Engineering:** Acts as an automated preliminary spatial scanning tool to jumpstart parametric CAD modeling (.STEP / .IGES) for rapid prototyping.

Important Engineering & Architectural Insights

1. Sequential Model GPU VRAM Management

Running multiple heavy generative models simultaneously would cause Out-Of-Memory (OOM) GPU crashes. To prevent this, the backend implements a sequential lifecycle strategy: **Load Model → Execute Inference → Explicitly Unload from Memory → Call torch.cuda.empty_cache()**. Furthermore, Hunyuan3D-2 utilizes cpu_offload, sequential_cpu_offload, attention_slicing, and vae_tiling to run smoothly even on consumer GPUs with 4GB-8GB VRAM.

2. Asynchronous Polling & Background Processing

Because 3D asset generation takes ~3 to 4 minutes, HTTP request timeouts are avoided by executing the pipeline asynchronously via FastAPI's BackgroundTasks. The frontend immediately receives a job_id upon upload and polls the status endpoint (GET /api/v1/pipeline/{job_id}/status) while rendering real-time progress steps.

3. Multi-Pass GroundingDINO Fallback Strategy

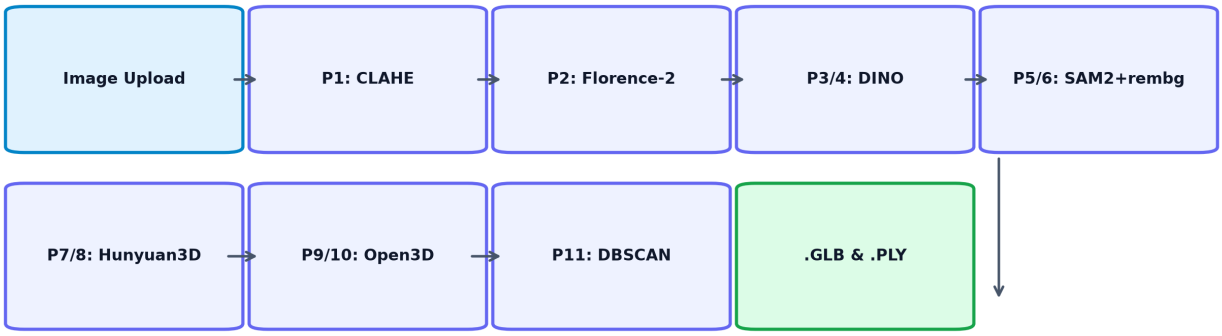
Low contrast or challenging lighting in user images can cause object detection failures. The system handles this with a 4-pass fallback mechanism: **Pass 1:** Standard detection (threshold 0.20) → **Pass 2:** Retry on CLAHE contrast-enhanced image → **Pass 3:** Lower threshold to 0.15 → **Pass 4:** Final low threshold (0.10).

4. Database Security & Local Fallback Mode

User security is ensured via **Supabase Row Level Security (RLS)**, allowing users to query only their own job history and output files. For standalone development, the backend automatically falls back to a **Local File Storage Mode** if Supabase environment variables are missing, storing job metadata in JSON files without crashing.

Comprehensive End-to-End Processing & Model Integration Flow

A detailed step-by-step breakdown of how the backend AI system handles pre-processing, prompt-free captioning, object detection, pixel segmentation, 3D mesh reconstruction, and point cloud clustering:



1. ■ Upload & Asynchronous Background Queueing

• **How it works:** When a user uploads an image via the Next.js interface, the FastAPI controller (upload_controller.py) receives the file, validates format/size, generates a unique job_id, and initiates a database record in Supabase.

• **Non-blocking processing:** The request immediately returns the job_id to the client while launching the 11-stage AI pipeline in the background using FastAPI BackgroundTasks (run.py).

2. ■ Phase 1: Image Analysis & CLAHE Enhancement

• **How it works:** The input image is converted to LAB color space. The backend measures the mean brightness and contrast standard deviation.

• **CLAHE (Contrast Limited Adaptive Histogram Equalization):** If the image has low contrast, OpenCV applies CLAHE to balance luminance across local image tiles. This sharpens edges and reveals object details in dark/shadowed areas without over-amplifying noise.

3. ■ Phase 2: Zero-Shot Auto Captioning (Florence-2)

• **How it works:** The enhanced image is passed to Microsoft's Florence-2 Vision-Language Model (microsoft/Florence-2-large).

• **Prompt-free execution:** Instead of forcing the user to manually write text prompts, Florence-2 automatically describes the scene (e.g. *'a ceramic coffee mug on a table'*) and extracts core keywords into a dot-separated prompt.

4. ■ Phase 3 & 4: Bounding Box Detection (GroundingDINO & Florence-2)

• **GroundingDINO Detection:** Uses text grounding to locate the target object's primary bounding box.

• **4-Pass Fallback Strategy:** If initial confidence is low (< 0.20), the system automatically retries with: (1) CLAHE-enhanced image, (2) Lowered threshold (0.15), (3) Final low threshold (0.10).

• **Florence-2 Part Detection:** Concurrently predicts sub-component bounding boxes (e.g. handle, body, base).

5. ✂■ Phase 5 & 6: SAM 2.1 Masking & rembg Cutout

• **SAM 2.1 (Segment Anything 2.1):** Meta's SAM 2.1 model takes the predicted bounding box coordinates as visual prompts and generates a high-precision binary pixel mask around object contours.

• **Background Removal (rembg):** The binary mask is combined with rembg (ONNX U-2-Net model) to strip away all background elements, exporting a transparent rgba.png cutout.

6. ■ Phase 7 & 8: 3D Mesh Generation & Texture Synthesis (Hunyuan3D-2)

• **Stage 1 (Geometry Diffusion):** Tencent's Hunyuan3D-2 generative model reconstructs a 3D geometry mesh from the transparent rgba.png image.

• **Stage 2 (Texture Synthesis):** Applies progressive texture diffusion to map high-resolution PBR surface textures onto the geometry, producing a ready-to-use .GLB 3D asset.

• **VRAM Memory Optimization:** Uses sequential model loading, CPU offloading, VAE slicing, and explicit torch.cuda.empty_cache() calls to prevent GPU Out-Of-Memory (OOM) errors.

7. ■ Phase 9, 10 & 11: Point Cloud Sampling & DBSCAN Clustering

• **Poisson Disk Sampling (Open3D):** Open3D loads the generated .GLB mesh and samples 100,000 surface coordinates while calculating surface normal vectors.

• **DBSCAN Spatial Clustering:** An unsupervised spatial clustering algorithm (DBSCAN) groups the 100,000 points into semantic structural clusters based on spatial density (eps=0.05, min_points=50). The result is exported as a segmented .PLY point cloud file.

8. ■ Real-Time Status Tracking & Persistence

• **State Updates:** After completing each stage, the backend updates artifacts_manager state.

• **Polling:** The Next.js frontend polls GET /api/v1/pipeline/status/{job_id} to render real-time progress bars and stage logs to the user.

Conclusion

The Automated Single-Image to 3D Asset and Point Cloud Generation System successfully addresses the challenges of traditional 3D modeling and point cloud extraction. By seamlessly integrating state-of-the-art generative AI models—Florence-2, GroundingDINO, SAM 2.1, Hunyuan3D-2, and Open3D—the system converts a single standard 2D image into production-ready 3D textured assets (.GLB) and semantically segmented point clouds (.PLY) within 3 to 4 minutes without requiring user prompt intervention.

Combined with a modern full-stack web application featuring Supabase user authentication, dynamic Three.js rendering, and persistent job history tracking, this project delivers an end-to-end scalable platform for automated 3D content creation.

Future Scope

- **Sparse Multi-View Integration:** Extend single-image generation to accept 2-3 sparse view inputs for improved geometry reconstruction of occluded rear surfaces.
- **Real-Time WebXR & Augmented Reality:** Integrate native WebXR features allowing users to directly project generated 3D models into real-world spaces via mobile AR browsers.
- **Mesh Optimization & Auto-Rigging:** Implement automatic polygon decimation, UV unwrapping, and skeletal auto-rigging for instant game-engine readiness.
- **Distributed GPU Task Queue:** Deploy Celery with Redis task queues to scale execution across multi-GPU server clusters for enterprise production concurrency.
- **CAD & Parametric Surface Export:** Convert output point clouds into standard CAD formats (.STEP / .IGES) for engineering and manufacturing workflows.